

pa7uiclf

Ciência da Computação

Revisão Geral: Paradigmas, Estruturas de Dados e
Java POO

19/09/2025

Paradigmas, Estruturas de Dados e Java POO



- Revisar o conceito de paradigma e suas variações (OO, Lógico, Funcional).
- Compreender as estruturas de listas encadeadas (Simples, Dupla, Circular) e suas aplicações.
- Entender os princípios da Recursividade e Ordenação de Listas.
- Consolidar a arquitetura, conceitos de POO e recursos modernos da linguagem Java

O que é um Paradigma de Programação?

Definição: É o **modelo conceitual** que guia como os programas são estruturados e executados.

Influência: O paradigma molda a forma como você pensa o problema, organiza o código e relaciona dados e operações.

Analogia (Atravessar um Rio):

- **OO:** Você cria barcos (objetos) e define como eles interagem.
- **Lógico:** Você descreve as regras de travessia, deixando o sistema encontrar a solução.
- **Funcional:** Você define funções puras que transformam o estado atual até alcançar o destino.

Comparativo dos Paradigmas (OO, Lógico, Funcional)

Paradigma	Como Vê o Programa	Foco Principal	Pensamento Central	Exemplo de Linguagem
Orientado a Objetos (OO)	Uma coleção de objetos que interagem	Entidades e interações	"Quais entidades existem? Como elas interagem?"	Java, Python, C++
Lógico	Um conjunto de fatos, regras e uma meta (objetivo) a ser provada	Relações e inferência	"Quais são as relações e regras? O sistema encontra o caminho."	Prolog
Funcional	Uma expressão que deve ser avaliada	Transformações de dados	Foco em funções puras (mesma saída para a mesma entrada, sem efeitos colaterais)	Haskell, Lisp, Elixir, Java (com Streams)

Listas Simplesmente Encadeadas (LSE)

Estrutura: Cada nó contém um valor (dado) e um ponteiro (referência) apenas para o **próximo nó**.

Características: Só permite andar em uma direção. É mais simples e ocupa menos memória do que a Duplamente Encadeada.

Aplicações:

- Implementação de Filas e Pilhas.
- Tabelas de Hash (para resolver colisões).
- Gerenciamento de blocos de memória livres em Sistemas Operacionais.
- Estruturas semelhantes a **Blockchain** (onde cada bloco aponta para o anterior).



Listas Duplamente Encadeadas (LDE)

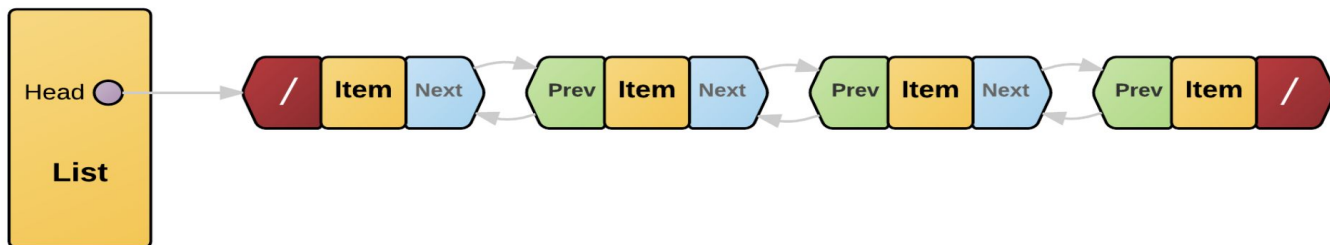
Estrutura: Cada nó contém um valor (dado), um ponteiro para o **próximo nó** e um ponteiro para o **nó anterior**.

Características: Permite navegar em **duas direções** (para frente e para trás).

Vantagens: A remoção de um nó é mais eficiente, pois o acesso ao nó anterior é direto, sem precisar percorrer a lista inteira.

Aplicações:

- Funções de **Undo/Redo** em editores de texto.
- Navegação em Navegadores (botões "voltar" e "avançar").
- Playlists em aplicativos de mídia.



Lista Encadeada Circular e Complexidade

Lista Encadeada Circular (LEC): Em vez de o último nó apontar para `NULL`, ele aponta de volta para o primeiro nó (a cabeça da lista), formando um "círculo".

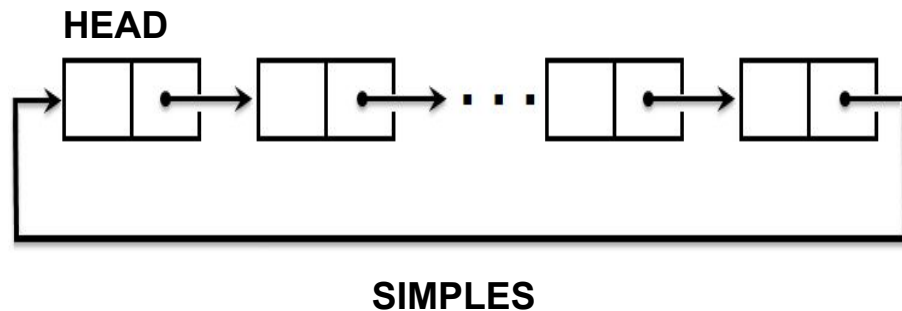
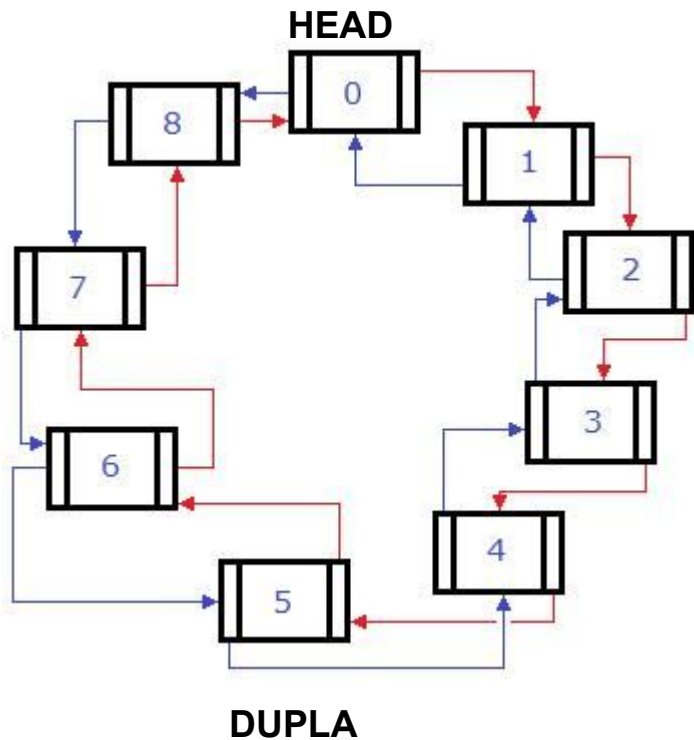
- Não há um "fim" definido; a travessia é contínua.
- A travessia exige uma condição de parada que verifica se o nó atual é novamente o nó inicial.

Análise de Algoritmos (Tempo de Acesso):

- Acesso por índice em listas encadeadas (LSE/LDE/LEC) é **$O(n)$** (cresce linearmente), pois é preciso percorrer os nós.
- $O(1)$ é constante (ex: acessar `lista`).
- $O(\log n)$ cresce devagar (ex: busca binária em vetor ordenado).

Quando Não Usar Listas Encadeadas: Quando há necessidade de acesso rápido e aleatório por índice, ou quando a memória é extremamente limitada (LDE tem overhead de 2 ponteiros).

Lista Encadeada Circular (Exemplo)



Ordenação de Listas Encadeadas

O Desafio: Algoritmos como Quick Sort e Merge Sort precisam de adaptações em listas encadeadas, pois **não há acesso aleatório** aos elementos.

Merge Sort (Melhor Escolha):

- É um algoritmo de divisão e conquista.
- É ideal para listas encadeadas porque a divisão e a junção são operações naturais e eficientes.
- A divisão é feita de forma eficiente, frequentemente usando a técnica dos ponteiros rápido e lento. **fast and slow pointer**
- A combinação é feita simplesmente ajustando os ponteiros `next` (não é preciso mover dados).

Quick Sort:

- Também é divisão e conquista, mas particiona a lista em elementos menores e maiores que um pivô.
- É mais complexo de implementar em listas encadeadas, pois a etapa de particionamento exige a manipulação de muitos ponteiros ou a criação de sub-listas.

Recursividade

Conceito: Uma técnica onde uma função chama a si mesma para resolver um problema, quebrando-o em versões menores e idênticas.

Partes Essenciais:

1. **Caso Base:** A condição de parada. Sem ele, ocorre o erro de *Stack Overflow* (estouro de pilha).
2. **Passo Recursivo:** A função chama a si mesma com uma versão simplificada do problema.

- **Vantagens:** Elegância, clareza e ideal para problemas com estrutura inerentemente recursiva.

- **Aplicações Chave:**

- Algoritmos de Divisão e Conquista (Merge Sort, Quick Sort).
- Travessia de Estruturas Recursivas (Árvores e Grafos).
- Em frameworks de persistência (como Hibernate/JPA), para carregar grafos de objetos relacionados recursivamente.

Recursividade

Qualidades e Vantagens da Recursividade

- **Elegância e Clareza do Código:** Para problemas que têm uma estrutura inerentemente recursiva (como a travessia de árvores, o cálculo de fatoriais ou a busca em grafos), a solução recursiva costuma ser mais curta, elegante e fácil de entender do que uma solução iterativa (com `for` ou `while`).
- **Resolução de Problemas Complexos:** A recursividade é ideal para resolver problemas complexos que podem ser divididos em subproblemas idênticos, como os algoritmos de ordenação **Merge Sort** e **Quick Sort**. Ela reflete a forma como pensamos em certos problemas, tornando a lógica mais intuitiva.
- **Menos Variáveis:** A recursividade muitas vezes elimina a necessidade de variáveis de controle complexas, como contadores e flags que seriam necessários em loops iterativos.

Java: Arquitetura e Fundamentos

Java: Linguagem orientada a objetos (POO), fortemente tipada, robusta e com gerenciamento automático de memória (Garbage Collector).

WORA: Write Once, Run Anywhere. Sua principal característica é a multiplataforma.

Processo de Execução:

1. Código Fonte (.java)
2. Compilador (`javac`)
3. Bytecode (.class) (código intermediário)
4. **JVM (Java Virtual Machine):** Interpreta/compila o bytecode para código de máquina.

Componentes da Plataforma:

- **JDK (Development Kit):** Inclui `javac`, JRE e ferramentas.
- **JRE (Runtime Environment):** Inclui JVM e bibliotecas necessárias para execução.

Gerenciamento de Memória e Tipos em Java



Áreas de Memória da JVM:

- **Stack (Pilha):** Armazena variáveis locais e parâmetros de métodos. É liberada automaticamente.
- **Heap:** Armazena objetos e arrays. É gerenciada pelo **Garbage Collector (GC)**.

Garbage Collection: É automático, liberando memória do *Heap* quando objetos não estão mais em uso.

Sistema de Tipos: Java é fortemente tipada.

- **Primitivos** (int, boolean, etc.).
- **Referência** (Classes, Arrays, Interfaces, Enums).

Nota: Estruturas como `ArrayList`, `Stream` e `StringBuilder` são classes de referência utilizadas para manipulação eficiente de dados no código Java.

Pilares da POO

- **Encapsulamento:** Oculta detalhes internos de implementação e controla o acesso aos dados via modificadores (getters/setters).
- **Herança:** Permite a reutilização de código e estabelece o relacionamento "é um" (palavra-chave `extends`).
- **Polimorfismo:** Permite que o mesmo método tenha comportamentos diferentes, seja por Sobrescrita (Override) ou Sobrecarga (Overload).

Abstração

Conceito: O processo de esconder detalhes complexos, expondo apenas as funcionalidades essenciais (ex: controle remoto).

Tipos de Abstração em Java:

- **Interface (100% abstrata):** Define um "**contrato**" que as classes devem cumprir. Uma classe pode implementar múltiplas interfaces.
- **Classe Abstrata (Abstração parcial):** Serve como uma "**base comum**" para classes relacionadas. Pode ter métodos abstratos e concretos. Uma classe só pode estender uma única classe abstrata.

Vantagem: Simplificação, reutilização, manutenibilidade e flexibilidade para o Polimorfismo.

Tratamento de Exceções

Java usa um sistema robusto para tratamento de erros.

Exceções Checked (Verificadas):

- Devem ser tratadas explicitamente pelo programador (`try-catch` ou `throws`).
- Representam condições **recuperáveis** (Ex: `IOException`, `FileNotFoundException`).
- Herda de `Exception`.

Exceções Unchecked (Não Verificadas):

- Não precisam ser tratadas explicitamente em tempo de compilação.
- Geralmente indicam **erros de programação** (Ex: `NullPointerException`, `ArrayIndexOutOfBoundsException`).
- Herda de `RuntimeException`.

Java Moderno e Collections

Collections Framework: Fornece estruturas prontas para gerenciar coleções de dados.

- **List:** Sequência ordenada (Ex: `ArrayList`).
- **Set:** Coleção sem duplicatas.
- **Map:** Armazena dados no formato Chave-Valor.

Recursos Modernos (Java 8+):

- **Lambda Expressions.**
- **Streams API:** Utilizada para processamento funcional de dados, permitindo código mais limpo e conciso

Comparativo de Linguagens (Contexto)

Linguagem	Especificidades Chave	Usos Comuns
Java	Orientada a Objetos, Portabilidade (JVM), Robusta, Fortemente Tipada.	Backend Corporativo, Aplicações Android, Big Data (Hadoop).
Python	Sintaxe clara e legível, Tipagem dinâmica.	Inteligência Artificial (ML/DL), Análise de Dados, Automação.
JavaScript	Executa no navegador (frontend), Assíncrona. Com Node.js, também backend.	Web (React, Angular), Backend (Node.js), Mobile.
C++	Baixo nível (acesso direto à memória), Alta performance.	Sistemas Operacionais, Jogos de alto desempenho (Unreal Engine).
Go (Golang)	Performance, Condição (Goroutines), Compilação rápida.	Microserviços, APIs, Infraestrutura de Nuvem.